SOFTWARE REQUIREMENTS SPECIFICATIONS

FOR

MULTI-VENDOR E-COMMERCE PLATFORM

Prepared By:

Sparsh Tyagi-202401100200308

Vaishnavi Rajawat-202401100200335

Shubham Patel-202401100200299

Varnit Tomar-202401100200341

Submitted To:

Mr. Rishabh Rathore Sir

(Faculty Mentor)

Department of Computer Science and Engineering

KIET Deemed to be University, Ghaziabad

Academic Year:2026-27

TABLE OF CONTENTS

1. Introduction 5
1.1 Purpose 5
1.2 Document Conventions 5
1.3 Intended Audience and Reading Suggestions 5
1.4 Product Scope 5
1.5 References 6
2. Overall Description 6
2.1 Product Perspective 6
2.2 Product Functions 6
2.3 User Classes and Characteristics 7
2.4 Operating Environment 7
2.5 Design and Implementation Constraints 7
2.6 User Documentation 8
2.7 Assumptions and Dependencies 8
3. External Interface Requirements 8
3.1 User Interfaces 8
3.2 Hardware Interfaces 9
3.3 Software Interfaces 10
3.4 Communications Interfaces 10
4. System Features 11
4.1 Authentication and Account Management 11
4.1.1 Description and Priority 11
4.1.2 Stimulus/Response Sequences 11
4.1.3 Functional Requirements 11
4.2 Vendor Onboarding and Management 12

© 2026 Project Team. Prepared for academic software engineering coursework.

4.2.1 Description and Priority 12
4.2.2 Stimulus/Response Sequences 12
4.2.3 Functional Requirements 12
4.3 Product and Catalog Management 13
4.3.1 Description and Priority 13
4.3.2 Stimulus/Response Sequences 13
4.3.3 Functional Requirements 13
4.4 Search, Discovery and Product View 14
4.4.1 Description and Priority 14
4.4.2 Stimulus/Response Sequences 14
4.4.3 Functional Requirements 14
4.5 Cart and Wishlist Management 14
4.5.1 Description and Priority 14
4.5.2 Stimulus/Response Sequences 14
4.5.3 Functional Requirements 15
4.6 Checkout and Payment Processing 15
4.6.1 Description and Priority 15
4.6.2 Stimulus/Response Sequences 15
4.6.3 Functional Requirements 15
4.7 Multi-Vendor Order and Fulfilment 16
4.7.1 Description and Priority 16
4.7.2 Stimulus/Response Sequences 16
4.7.3 Functional Requirements 16
4.8 Inventory Management 17
4.8.1 Description and Priority 17
4.8.2 Stimulus/Response Sequences 17
4.8.3 Functional Requirements 17
4.9 Shipping, Returns and Refunds 17
4.9.1 Description and Priority 17
4.9.2 Stimulus/Response Sequences 17
4.9.3 Functional Requirements 18
4.10 Reviews and Ratings 18
4.10.1 Description and Priority 18
4.10.2 Stimulus/Response Sequences 18
4.10.3 Functional Requirements 18
4.11 Coupons, Commission and Settlements 19
4.11.1 Description and Priority 19
4.11.2 Stimulus/Response Sequences 19
4.11.3 Functional Requirements 19
4.12 Notifications 19

© 2026 Project Team. Prepared for academic software engineering coursework.

4.12.1 Description and Priority 19
4.12.2 Stimulus/Response Sequences 19
4.12.3 Functional Requirements 20
4.13 Administration and Reporting 20
4.13.1 Description and Priority 20
4.13.2 Stimulus/Response Sequences 20
4.13.3 Functional Requirements 20
5. Other Nonfunctional Requirements 21
5.1 Performance Requirements 21
5.2 Safety Requirements 21
5.3 Security Requirements 21
5.4 Software Quality Attributes 22
5.5 Business Rules 23
6. Other Requirements 23
6.1 Database Requirements 23
6.2 Internationalization Requirements 24
6.3 Legal, Compliance and Licensing Requirements 24
6.4 Reuse Objectives 24
Appendix A: Glossary 24
Appendix B: Analysis Models 25
B.1 System Architecture / Component Diagram 25
B.2 Use Case Diagram 26
B.3 Entity-Relationship Diagram 27
B.4 Order Status State Diagram 28
Appendix C: To Be Determined List 28
Future Scope (Beyond Version 1.0) 29

© 2026 Project Team. Prepared for academic software engineering coursework.

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional, non-functional, interface, data and quality requirements for Version 1.0 of the Multi-Vendor E-Commerce Platform. It specifies the entire system: the customer storefront, the vendor dashboard and the administrator dashboard, together with the backend services and data store that support them. This document is written to IEEE 830 / IEEE 29148 conventions and is the authoritative baseline for design, implementation, testing and acceptance of the platform.

1.2 Document Conventions

Each requirement is assigned a unique, permanent identifier of the form REQ-<AREA>-<NNN> (functional) or NFR-<NNN> (non-functional); identifiers are never reused even if a requirement is later removed. Requirement statements use “shall” to denote a mandatory, testable requirement and “should” to denote a strongly recommended but non-mandatory characteristic. Every requirement carries an explicit priority of High, Medium or Low; unless stated otherwise, a lower-level requirement inherits the priority of the feature under which it appears. Section numbering follows the standard IEEE SRS outline (Introduction, Overall Description, External Interface Requirements, System Features, Other Nonfunctional Requirements, Other Requirements, Appendices). Headings use Heading 1/2/3 Word styles so that the Table of Contents updates automatically.

1.3 Intended Audience and Reading Suggestions

This document is intended for the project team and developers, system analysts and designers, test engineers, DevOps/deployment personnel, evaluating faculty, and other project stakeholders.

- Developers and system designers should read Sections 3, 4, 5, 7 and Appendix B in full for interface, feature-level and data detail.
- Test engineers should focus on Sections 4 (Stimulus/Response and Functional Requirements), 5.5 (Business Rules) and the acceptance criteria referenced from each feature.
- Project managers and evaluators may read Sections 1 and 2 for scope and context, then skim Section 4 headings for feature coverage.
- New readers are encouraged to read sequentially from Section 1 through Section 2 before jumping to a specific feature in Section 4.

1.4 Product Scope

The Multi-Vendor E-Commerce Platform is a new, self-contained, web-based marketplace that allows multiple independent vendors to sell products to customers through a single storefront. Its purpose is to demonstrate a realistic, end-to-end marketplace: vendor onboarding and moderation, product catalogue management, multi-vendor cart and checkout, split order fulfilment, payments, returns/refunds, reviews, coupons, commission and vendor settlement, and administrative oversight and reporting.

The platform's objectives are to (a) provide a secure and usable marketplace for customers and vendors, (b) support the complete order lifecycle from cart to settlement, (c) maintain accurate financial and inventory data, and (d) provide a modular architecture that can be extended with the advanced capabilities listed in Section 6 and the Future Scope discussion in Appendix C. Native mobile apps, automated KYC/banking integration and enterprise logistics are out of scope for Version 1.0.

© 2026 Project Team. Prepared for academic software engineering coursework.

1.5 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering.
- OWASP Top 10 (owasp.org) — baseline web application security risks referenced in Section 5.3.
- WCAG 2.1 (w3.org/TR/WCAG21) — accessibility guidance referenced in Section 5.4.
- Payment gateway sandbox documentation (e.g., Razorpay or Stripe) — for integration and webhook behaviour referenced in Section 3.3.
- This SRS template is adapted from the IEEE-conformant SRS template originally developed by Karl E. Wiegers.

2. Overall Description

2.1 Product Perspective

The Multi-Vendor E-Commerce Platform is a new, self-contained product; it is not a follow-on to, or replacement for, an existing system, and it does not migrate data from a legacy system. It is composed of three role-specific presentation applications (customer storefront, vendor dashboard, administrator dashboard) that share a common backend API, business-logic layer and relational database, and that integrate with a small number of external services (payment gateway, email/notification provider, object storage and, optionally, a shipping API). Figure 1 (Appendix B.1) shows the major components of the system and their interconnections.

2.2 Product Functions

The major functions the product must perform, detailed fully in Section 4, are summarized below:

- Authentication and role-based authorization for Customer, Vendor and Administrator roles
- Customer account and address management
- Vendor onboarding, verification and approval
- Product catalog, category and moderation management
- Search, filtering and sorting of products
- Cart and Wishlist management, including multi-vendor carts
- Checkout, coupon application and payment processing
- Multi-vendor order creation and vendor-specific sub-order fulfilment
- Inventory tracking and stock validation
- Shipping/tracking, cancellation, return and refund handling
- Product reviews and ratings
- Notifications across account, order, payment and moderation events
- Commission calculation and vendor settlement tracking
- Administrative moderation, monitoring and analytics/reporting

© 2026 Project Team. Prepared for academic software engineering coursework.

2.3 User Classes and Characteristics

Three user classes are anticipated. They are distinguished primarily by privilege level and by the subset of product functions each is entitled to use; all three are considered equally important to satisfy since the marketplace cannot function without any one of them.

User Class	Characteristics	Relative Importance
Customer	General public with typical web/mobile literacy. No training required. Frequency of use: high (daily/weekly browsing and periodic purchases). Uses discovery, cart, checkout, order tracking and review functions.	High — primary revenue-generating user
Vendor	Small-to-medium business owner or staff, basic-to-moderate digital literacy. Frequency of use: regular (daily catalog/order management). Uses onboarding, catalog, inventory, orderprocessing and earnings functions.	High — supplies the marketplace's inventory
Administrator	Trusted platform operations staff with training on moderation and financial oversight. Frequency of use: regular. Uses approval, moderation, monitoring, configuration and reporting functions.	High — required for trust and compliance

2.4 Operating Environment

The platform operates as a three-tier web application. Client software is any modern desktop, tablet or mobile web browser (Chrome, Edge, Firefox or Safari, current or immediately preceding major version); no browser plug-ins are required. Backend services run inside Docker containers on a Linux-based host, using the Node.js LTS runtime, and communicate with a PostgreSQL relational database and an optional Redis instance for caching and lightweight queues. The application is deployed to a cloud-hosted environment (for example AWS, Azure, GCP, or an equivalent academic/free-tier host) with HTTPS termination at the edge, and coexists with the external services listed in Section 3.3.

2.5 Design and Implementation Constraints

- The system must enforce strict, server-side role-based authorization for every protected operation.
- Payment data must be handled through a compliant external payment gateway; the platform must not store sensitive card credentials.
- Third-party integrations (payment, email, storage, shipping) may be unavailable during development and therefore require sandbox/mock support.
- Courier GPS tracking, KYC identity verification and real vendor bank payouts may be simplified or simulated in the academic baseline (see Appendix C).
- The technology stack is constrained to the components listed in Section 3.3 to keep the team's skill set and delivery timeline realistic.
- The implementation should remain modular so individual services can be replaced without redesigning the entire application.
- Academic project timeline and team size constrain implementation breadth; scope is prioritized by the priority levels in Section 4.

© 2026 Project Team. Prepared for academic software engineering coursework.

2.6 User Documentation

The following user-facing documentation components will be delivered alongside the software:

- A Customer Quick-Start Guide covering registration, browsing, checkout and order tracking (delivered as an in-app help page or PDF).
- A Vendor Onboarding and Operations Guide covering application submission, product listing, order processing and settlements (in-app help / PDF).
- An Administrator Operations Guide covering approvals, moderation, coupon and commission configuration, and report usage (PDF / internal wiki).
- Inline contextual help (tooltips and field-level guidance) on complex forms such as product creation and checkout.
- API reference documentation for backend endpoints, generated from code annotations, for the development team.

2.7 Assumptions and Dependencies

- Users have an internet connection and a supported browser.
- Vendors provide accurate product and business information.
- Administrators review vendor and product submissions in a timely manner.
- The external payment provider is available and returns reliable success/failure status for payment operations.
- Email/notification services are available when configured.
- The project depends on PostgreSQL, optional Redis, a payment gateway sandbox, an email/notification provider, cloud/object storage, cloud hosting with DNS/HTTPS, and a sourcecontrol/CI-CD platform, all of which are assumed to remain available throughout development.
- Real courier GPS tracking and real vendor bank payouts are assumed to be simulated in the academic version; the project would be affected if a genuine integration were later required.

3. External Interface Requirements

3.1 User Interfaces

The platform exposes three role-specific web interfaces — the Customer Storefront, the Vendor Dashboard and the Administrator Dashboard — sharing a common visual design system and component library. All interfaces are responsive across desktop, tablet and mobile viewports (see NFR-019) and follow the accessibility practices in Section 5.4.

Interface	Logical Characteristics

© 2026 Project Team. Prepared for academic software engineering coursework.

Customer Storefront	Home/discovery, category and search results pages, product detail page, cart, checkout, order history/tracking, profile and address management, review submission, and notifications. Every screen provides a persistent header with search, cart and account access.
Vendor Dashboard	Onboarding-status screen, product list/create/edit forms, inventory view, incoming-order queue with status actions, earnings/settlement summary, and vendor profile settings. A left-hand navigation panel is present on all screens.
Admin Dashboard	Vendor and product moderation queues, user/vendor management, order/payment/refund monitoring, category and coupon management, review moderation, and reporting dashboards. Standard buttons (Approve, Reject, Suspend, Export) appear consistently wherever applicable.

- Destructive or financially significant actions (cancel order, approve refund, deactivate vendor) require an explicit confirmation dialog.
- Form validation errors are displayed inline, next to the offending field, in plain language, using a consistent red-text/icon convention.
- Loading, empty and error states are explicitly designed for all data-driven views (product lists, order lists, dashboards).
- Standard keyboard navigation (Tab order, Enter to submit, Esc to close dialogs) is supported on all forms.
- A detailed visual UI specification (wireframes/mock-ups) is maintained as a separate design artifact and is not duplicated in this SRS.

3.2 Hardware Interfaces

The baseline system has no direct or specialised hardware interface requirements. Client devices are limited to standard consumer hardware (desktop, laptop, tablet, smartphone) capable of running a supported browser; the platform does not require barcode scanners, POS terminals, card readers or other peripherals in Version 1.0. Server-side, the platform is hardware-agnostic and runs on standard virtualised cloud compute; the hosting provider is responsible for underlying hardware and network management. Should a future release introduce a native mobile application, camera access for barcode/QR scanning is anticipated as a hardware interface (see Appendix C).

© 2026 Project Team. Prepared for academic software engineering coursework.

3.3 Software Interfaces

Component	Name / Version	Purpose / Data Exchanged
Frontend framework	Next.js / React with TypeScript	Renders the three role-specific web applications; consumes backend REST APIs over JSON.
Backend runtime	Node.js (LTS) with TypeScript	Hosts REST API services implementing authentication, business logic and orchestration.
Database	PostgreSQL (current stable release)	Primary relational data store for all entities in Section 7.
ORM	Prisma (or equivalent)	Maps application entities to relational tables; manages migrations.
Cache / Queue	Redis (optional)	Session/response caching and lightweight background job queues (e.g., notification dispatch).
Payment gateway	Razorpay or Stripe (sandbox/production)	Payment authorization, capture and refund; sends webhook events for payment status.
Email/notification provider	SMTP or transactional email API (e.g., SendGrid)	Delivers account, order, payment and moderation notifications.
Object storage	Cloudinary or Amazon S3 (or equivalent)	Stores and serves product images and vendor logos.
Shipping API (optional)	Third-party courier API	Retrieves shipment status/tracking where integrated.
Containerization	Docker / Docker Compose	Packages and runs each service consistently across environments.
CI/CD	GitHub Actions (or equivalent)	Automates build, test and deployment pipelines.

3.4 Communications Interfaces

- All client-server communication uses HTTPS in production; REST APIs exchange data using JSON with consistent, documented status codes and error payload structures.
- Authentication information is transmitted using secure mechanisms appropriate to the chosen design (e.g., JWT bearer tokens or secure, HTTP-only session cookies).
- Transactional email is sent via the configured email/notification provider using the provider's authenticated API or SMTP-over-TLS.
- Webhooks received from external services (payment gateway, shipping API) are cryptographically verified before they are allowed to update internal state.

© 2026 Project Team. Prepared for academic software engineering coursework.

- Where real-time updates are needed (e.g., order status), the system may use polling or a lightweight push mechanism (e.g., WebSocket) with a documented message format.

- Network security follows standard practice: TLS 1.2+ for all external endpoints, and no communication of card data outside the payment gateway's hosted fields/SDK.

4. System Features

This section organizes the product's functional requirements by system feature — the major services the product provides. Each feature states its description and priority, the stimulus/response sequences that characterize its use, and its itemized, uniquely identified functional requirements. Requirements use “shall” for mandatory, testable behaviour, per the conventions in Section 1.2.

4.1 Authentication and Account Management

4.1.1 Description and Priority

Allows customers and vendor applicants to register, authenticate, recover access and manage their own profile and delivery addresses, and assigns the correct role-based permissions to every authenticated session. This feature underlies every other feature in the system and is therefore rated High priority. Priority: High.

4.1.2 Stimulus/Response Sequences

- User submits registration details → system validates uniqueness/format, creates the account, and (if configured) sends a verification email.
- User submits login credentials → system authenticates and issues a session/token scoped to the user's role.
- User selects “Forgot password” and submits their email → system sends a verified recovery link/token; user submits a new password → system updates the credential and invalidates existing sessions.
- User edits profile or address fields and submits → system validates and persists the change.
- User selects “Logout” → system invalidates the active session/token.

4.1.3 Functional Requirements

REQ-AUTH-01: The system shall allow a new customer or vendor applicant to create an account using required registration information (name, email, password, and role-relevant fields).

REQ-AUTH-02: The system shall authenticate registered users using valid credentials and reject invalid ones with a generic error that does not reveal which field was incorrect.

REQ-AUTH-03: The system shall support logout and secure session/token invalidation.

REQ-AUTH-04: The system shall support password reset through a verified recovery mechanism (e.g., time-limited email link).

REQ-AUTH-05: The system shall support email verification where configured, and shall restrict sensitive actions until verification is complete if so configured.

© 2026 Project Team. Prepared for academic software engineering coursework.

REQ-AUTH-06: The system shall assign an appropriate role to an authenticated user and enforce role-based access control on every protected operation.

REQ-AUTH-07: The system shall allow users to update permitted profile information and change their password after re-authentication.

REQ-AUTH-08: The system shall allow customers to add, update and remove delivery addresses, with at least one address markable as default.

REQ-AUTH-09: The system shall support multi-factor authentication (MFA) for administrator and vendor accounts as an optional or enforced control.

REQ-AUTH-10: The system shall lock or throttle an account after a configured number of consecutive failed login attempts and shall log the event.

4.2 Vendor Onboarding and Management

4.2.1 Description and Priority

Allows a user to apply to become a vendor, and allows an administrator to review, approve, reject, suspend or reactivate that vendor, gating a vendor's ability to publish products on approval status. Priority: High.

4.2.2 Stimulus/Response Sequences

- Applicant submits business/profile information → system creates a vendor application in Pending status and notifies administrators.
- Administrator opens a Pending application and selects Approve or Reject → system updates status and notifies the applicant.
- Administrator selects Suspend on an active vendor → system immediately unpublishes that vendor's active products.
- Approved vendor logs in → system displays a dashboard summarizing products, orders, inventory and earnings.

4.2.3 Functional Requirements

REQ-VEND-01: The system shall allow a user to submit a vendor application containing required business/profile information.

REQ-VEND-02: The system shall maintain vendor application status such as Pending, Approved, Rejected or Suspended.

REQ-VEND-03: The administrator shall be able to review and approve or reject vendor applications.

REQ-VEND-04: Only approved vendors shall be allowed to publish active products.

REQ-VEND-05: The administrator shall be able to suspend or reactivate vendors; suspension shall immediately unpublish that vendor's active products.

REQ-VEND-06: The system shall provide an approved vendor with a dashboard summarizing products, orders, inventory and earnings.

REQ-VEND-07: The system shall allow a vendor to maintain a public storefront profile including business name, logo, description and policies.

© 2026 Project Team. Prepared for academic software engineering coursework.

4.3 Product and Catalog Management

4.3.1 Description and Priority

Allows approved vendors to create and maintain product listings and allows administrators to moderate listings and organize the category taxonomy that structures the catalog. Priority: High.

4.3.2 Stimulus/Response Sequences

- Vendor submits a new product form → system validates required fields and stores the listing in the configured moderation state.
- Administrator reviews a submitted product and selects Approve or Reject → system updates product status and notifies the vendor.
- Vendor edits or deactivates their own product → system persists the change and re-applies moderation rules if required.
- Administrator creates or edits a category → system updates the taxonomy used by search and browse.

4.3.3 Functional Requirements

REQ-CAT-01: Vendors shall be able to create product listings with name, description, SKU, price, discount, category and stock information.

REQ-CAT-02: Vendors shall be able to upload product images through the configured storage mechanism.

REQ-CAT-03: Vendors shall be able to edit and deactivate their own product listings.

REQ-CAT-04: The system shall support product status values including Draft, Submitted, Pending Approval, Approved, Published, Inactive and Out of Stock as applicable.

REQ-CAT-05: Administrators shall be able to approve, reject, deactivate or moderate product listings.

REQ-CAT-06: Administrators shall be able to create, update, deactivate and organize categories and subcategories.

REQ-CAT-07: The system shall prevent a vendor from modifying products belonging to another vendor.

REQ-CAT-08: The system shall support product variants (e.g., size, colour) where applicable, each with its own SKU and stock count.

REQ-CAT-09: The system shall maintain a version/audit history of significant product changes (price, stock, status).

© 2026 Project Team. Prepared for academic software engineering coursework.

4.4 Search, Discovery and Product View

4.4.1 Description and Priority

Allows customers to find and evaluate products through browsing, keyword search, filters, sorting and a detailed product view. Priority: High.

4.4.2 Stimulus/Response Sequences

- Customer enters a keyword and submits → system returns matching active products ranked by relevance.
- Customer applies a filter (category, price, vendor, rating, availability) → system narrows the result set accordingly.
- Customer selects a sort option → system reorders the result set without losing active filters.
- Customer opens a product → system displays full product detail, price, availability, vendor information, ratings and reviews.

4.4.3 Functional Requirements

REQ-DISC-01: Customers shall be able to browse active products and categories.

REQ-DISC-02: Customers shall be able to search products by keywords and relevant searchable attributes.

REQ-DISC-03: The system shall support filtering by category, price, vendor, rating and availability where applicable.

REQ-DISC-04: The system shall support sorting by price, rating, popularity or recency where applicable.

REQ-DISC-05: The product detail view shall display product information, price, availability, vendor information, ratings and reviews where available.

REQ-DISC-06: The system shall present related or recommended products on the product detail page.

4.5 Cart and Wishlist Management

4.5.1 Description and Priority

Allows a customer to accumulate products they intend to purchase, including products from multiple vendors in one cart, and to save products of interest to a wishlist for later. Priority: High.

4.5.2 Stimulus/Response Sequences

- Customer selects Add to Cart on a product → system adds/increments the item and revalidates stock.
- Customer updates a cart line's quantity or removes it → system recalculates the cart subtotal.

© 2026 Project Team. Prepared for academic software engineering coursework.

- Customer selects Add to Wishlist → system stores the reference against the customer's account.
- Customer moves a wishlist item to the cart → system validates current availability before adding.

4.5.3 Functional Requirements

REQ-CART-01: Authenticated customers shall be able to add products to a shopping cart.

REQ-CART-02: Customers shall be able to update item quantities and remove items from the cart.

REQ-CART-03: The system shall validate current product availability and stock before final order placement.

REQ-CART-04: The system shall allow a cart to contain products belonging to multiple vendors.

REQ-CART-05: Customers shall be able to add or remove products from a wishlist.

REQ-CART-06: Customers shall be able to move eligible wishlist items to the cart.

REQ-CART-07: A guest (unauthenticated) user shall be able to maintain a temporary cart that merges into the account cart upon login.

4.6 Checkout and Payment Processing

4.6.1 Description and Priority

Guides an authenticated customer through address selection, order summary, coupon application and payment, and ensures that only a verified successful payment results in a confirmed order. Priority: High.

4.6.2 Stimulus/Response Sequences

- Customer proceeds to checkout → system validates cart availability and presents address and order-summary steps.
- Customer applies a coupon code → system validates eligibility/expiry and recalculates the total.
- Customer confirms and selects a payment method → system initiates payment through the configured gateway.
- Gateway returns a success or failure result (directly or via webhook) → system records the transaction and confirms or fails the order accordingly.

4.6.3 Functional Requirements

REQ-CHK-01: The system shall provide a checkout process that captures shipping address, order summary and available delivery information.

REQ-CHK-02: The system shall calculate subtotal, applicable discount, shipping charge, tax if configured and final payable amount.

REQ-CHK-03: The system shall allow eligible coupons to be applied and validated before order confirmation.

© 2026 Project Team. Prepared for academic software engineering coursework.

REQ-CHK-04: The system shall initiate payment through the configured payment gateway.

REQ-CHK-05: The system shall record payment transaction identifiers and payment status without storing prohibited sensitive payment credentials.

REQ-CHK-06: A successful payment shall trigger order confirmation according to the configured transaction flow; a failed payment shall not create a falsely confirmed order.

REQ-CHK-07: The system shall support at least one alternate payment mode (e.g., Cash on Delivery) if enabled by configuration.

4.7 Multi-Vendor Order and Fulfilment

4.7.1 Description and Priority

Splits a successfully paid multi-vendor checkout into vendor-specific sub-orders, and allows each vendor and the customer to track and act on order status through its lifecycle. Priority: High.

4.7.2 Stimulus/Response Sequences

- Payment succeeds for a checkout containing items from multiple vendors → system creates one parent order and one vendor order per vendor group.
- Vendor opens their order queue → system shows only vendor orders containing that vendor's items.
- Vendor updates a vendor order's status (e.g., to Shipped) → system records the transition and notifies the customer.
- Customer requests cancellation of an eligible order → system validates the current status against policy before cancelling.

4.7.3 Functional Requirements

REQ-ORD-01: The system shall create an order for a successful checkout transaction.

REQ-ORD-02: The system shall split a multi-vendor purchase into vendor-specific sub-orders linked to the parent customer order.

REQ-ORD-03: Each vendor order shall contain only the items owned by the relevant vendor.

REQ-ORD-04: The system shall maintain order statuses including Placed, Confirmed, Processing, Shipped, Delivered, Cancelled, Returned and Refunded as applicable.

REQ-ORD-05: Customers shall be able to view current and historical orders.

REQ-ORD-06: Vendors shall be able to view and process only orders containing their products.

REQ-ORD-07: Customers shall be able to cancel eligible orders based on order status and business rules.

REQ-ORD-08: The system shall maintain an auditable status history for significant order state changes.

REQ-ORD-09: The system shall generate a unique, human-readable order reference number for each parent order and vendor order.

© 2026 Project Team. Prepared for academic software engineering coursework.

4.8 Inventory Management

4.8.1 Description and Priority

Keeps stock quantities accurate as orders, cancellations and returns occur, and prevents customers from ordering more than is available. Priority: High.

4.8.2 Stimulus/Response Sequences

- Vendor updates a stock quantity → system persists the new value and re-evaluates out-of-stock status.
- Customer attempts to order a quantity exceeding available stock → system blocks the action with a clear message.
- An order reaches the configured successful-order event → system decrements the corresponding inventory.
- An eligible cancellation or return is approved → system restores the corresponding inventory.

4.8.3 Functional Requirements

REQ-INV-01: Vendors shall be able to view and update inventory quantities for their products.

REQ-INV-02: The system shall prevent order placement when requested quantity exceeds available stock.

REQ-INV-03: The system shall decrement available inventory after the configured successful order event.

REQ-INV-04: The system shall restore inventory when an eligible cancellation or return requires stock restoration.

REQ-INV-05: The system shall indicate products as out of stock when available quantity reaches zero.

REQ-INV-06: The system may generate low-stock alerts based on a vendor-configured or platformdefined threshold.

4.9 Shipping, Returns and Refunds

4.9.1 Description and Priority

Tracks shipment progress for a vendor order and manages the return/refund request, review and settlement process for eligible orders. Priority: High.

4.9.2 Stimulus/Response Sequences

- Vendor marks a vendor order as Packed/Shipped → system updates shipment status and, where available, tracking information.

© 2026 Project Team. Prepared for academic software engineering coursework.

- Customer requests a return or refund on an eligible order → system validates the eligibility window and routes the request.

- Vendor or administrator approves or rejects the request → system updates request status and, on approval, initiates the refund.
- Refund completes at the payment gateway → system updates refund and order financial records.

4.9.3 Functional Requirements

REQ-SHIP-01: The system shall store the delivery address associated with an order.

REQ-SHIP-02: The system shall maintain shipment status and tracking information when available.

REQ-SHIP-03: The system shall support shipment statuses such as Confirmed, Packed, Shipped, Out for Delivery and Delivered where applicable.

REQ-SHIP-04: Customers shall be able to request an eligible return or refund.

REQ-SHIP-05: The appropriate vendor or administrator shall be able to review and approve/reject eligible return/refund requests.

REQ-SHIP-06: The system shall update refund status and associated transaction/order records when a refund is initiated or completed.

4.10 Reviews and Ratings

4.10.1 Description and Priority

Allows customers who purchased a product to share feedback, and allows administrators to moderate reviews that violate platform rules. Priority: Medium.

4.10.2 Stimulus/Response Sequences

- Customer with an eligible purchase submits a rating and comment → system validates eligibility and publishes the review.
- Vendor opens a review of their product and submits a public response → system attaches the response to the review.
- Administrator flags or removes a review that violates platform rules → system unpublishes the review.

4.10.3 Functional Requirements

REQ-REV-01: Customers shall be able to submit a review and rating for eligible purchased products.

REQ-REV-02: The system shall prevent unauthorized users from submitting reviews.

REQ-REV-03: Administrators shall be able to moderate or remove reviews that violate platform rules.

REQ-REV-04: Vendors shall be able to publicly respond to a review of their own product.

© 2026 Project Team. Prepared for academic software engineering coursework.

4.11 Coupons, Commission and Settlements

4.11.1 Description and Priority

Let’s administrators configure discount coupons and commission rules, and computes vendor earnings and settlement amounts consistently from completed vendor orders. Priority: High.

4.11.2 Stimulus/Response Sequences

- Administrator creates or edits a coupon → system stores validity, limits, expiry and applicable conditions.
- Customer applies a coupon at checkout → system validates it against current rules before applying the discount.
- A vendor order reaches the configured successful state → system calculates platform commission and vendor earnings for that order.
- Settlement cycle closes → system aggregates a vendor's eligible earnings into a settlement record and statement.

4.11.3 Functional Requirements

REQ-COM-01: Administrators shall be able to create, update, activate and deactivate coupons.

REQ-COM-02: The system shall validate coupon eligibility, limits, expiry and applicable conditions before applying a discount.

REQ-COM-03: The system shall calculate platform commission according to configured commission rules.

REQ-COM-04: The system shall calculate vendor earnings after applicable commission and other configured deductions.

REQ-COM-05: The system shall maintain vendor settlement status and associated amounts.

REQ-COM-06: The system shall generate a periodic settlement statement per vendor for a configurable settlement cycle.

4.12 Notifications

4.12.1 Description and Priority

Keeps customers, vendors and administrators informed of relevant account, order, payment, shipping, approval and refund events through configured channels. Priority: Medium.

4.12.2 Stimulus/Response Sequences

- A qualifying event occurs (e.g., order confirmed) → system generates and dispatches a notification to the relevant party.
- Notification dispatch fails (e.g., provider outage) → system retries according to a backoff policy and logs the failure if retries are exhausted.

© 2026 Project Team. Prepared for academic software engineering coursework.

4.12.3 Functional Requirements

REQ-NOT-01: The system shall generate relevant notifications for account, order, payment, shipping, approval and refund events.

REQ-NOT-02: Customers shall receive order and payment status notifications through configured channels.

REQ-NOT-03: Vendors shall receive important product approval, order and inventory notifications through configured channels.

REQ-NOT-04: Administrators shall be alerted to important operational events such as new vendor applications or exceptions where configured.

4.13 Administration and Reporting

4.13.1 Description and Priority

Gives administrators the tools to manage marketplace entities, monitor operational and financial activity, and view summary reports; gives vendors visibility into their own performance. Priority: High.

4.13.2 Stimulus/Response Sequences

- Administrator opens the customer or vendor management screen → system lists accounts with search/filter and available actions.
- Administrator opens the operations dashboard → system displays current order, payment, return and refund summaries.
- Vendor opens their sales summary → system displays orders, sales and earnings scoped to that vendor.
- Administrator exports a report → system produces a downloadable file (e.g., CSV) of the underlying data.

4.13.3 Functional Requirements

REQ-ADM-01: Administrators shall be able to view and manage customers.

REQ-ADM-02: Administrators shall be able to view and manage vendors and their status.

REQ-ADM-03: Administrators shall be able to monitor orders, payments, returns and refunds.

REQ-ADM-04: Administrators shall be able to manage categories, products, coupons and reviews.

REQ-ADM-05: The system shall provide summary dashboards for orders, sales, vendors, customers and platform commission.

REQ-ADM-06: Vendors shall be able to view sales, order and earnings summaries relevant to their own account.

REQ-ADM-07: The system shall record sufficient data to support operational and financial reports.

REQ-ADM-08: The system shall allow administrators to export report data (e.g., CSV) for offline analysis.

© 2026 Project Team. Prepared for academic software engineering coursework.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

Performance requirements below apply under the expected academic-project load (a small number of concurrent users) and should guide sensible design choices (indexing, pagination, caching) rather than force premature optimization.

- NFR-001 [Medium]: For expected academic-project load, the system should return 95% of normal API requests within 2 seconds under normal operating conditions.
- NFR-002 [Medium]: Search and catalog pages should provide results within an acceptable interactive response time (target: under 2 seconds) under expected load.
- NFR-003 [Medium]: Database access patterns shall use appropriate indexing and pagination for large result sets (e.g., product listings, order history).
- NFR-004 [Low]: Checkout and payment-initiation actions should complete their initial round trip within 3 seconds under normal conditions, excluding external gateway latency.

5.2 Safety Requirements

“Safety” in this context refers to protection against loss, damage or harm arising from incorrect financial or inventory behaviour, since the platform does not control physical equipment.

- NFR-005 [High]: The system shall never confirm an order as paid unless the configured payment verification process has indicated success (see BR-04), to prevent financial loss from falsely confirmed orders.
- NFR-006 [High]: The system shall never allow inventory to go negative as a result of order processing, to prevent vendors from being obligated to fulfil unavailable stock.
- NFR-007 [Medium]: A refund shall never be permitted to exceed the original captured payment amount for the associated order (see BR-15).
- NFR-008 [Medium]: Destructive administrative actions (e.g., permanently deleting a vendor or product) shall be disabled in favour of deactivation/soft-delete, to prevent irreversible data loss (see Section 7.2).

5.3 Security Requirements

Security requirements address authentication, authorization, data protection and defence against common web application risks, informed by the OWASP Top 10 referenced in Section 1.5.

- NFR-009 [High]: Passwords shall be stored using a secure one-way password hashing mechanism (e.g., bcrypt/argon2); passwords shall never be stored or logged in plain text.
- NFR-010 [High]: Protected APIs shall enforce authentication and role-based authorization serverside, regardless of what the client interface displays or hides.
- NFR-011 [Medium]: Input validation and output encoding shall be used to reduce common injection and cross-site scripting (XSS) risks.
- NFR-012 [Medium]: Production communication shall use HTTPS/TLS 1.2 or higher for all clientserver and server-to-integration traffic.

© 2026 Project Team. Prepared for academic software engineering coursework.

- NFR-013 [Medium]: Secrets and credentials shall not be hard-coded in source code; they shall be supplied through environment-specific configuration or a secrets manager.
- NFR-014 [Medium]: Sensitive administrative operations (approvals, refunds, suspensions, configuration changes) shall be auditable through logs including actor, timestamp and action.
- NFR-015 [Medium]: The platform shall not store prohibited raw payment card credentials; card data is handled exclusively by the payment gateway's hosted fields or SDK.
- NFR-016 [Low]: Administrator and vendor accounts should support session timeout after a configurable period of inactivity.

5.4 Software Quality Attributes

The following quality attributes are important to this product; each is stated to be as specific and verifiable as practical for an academic-scale system.

- **Availability** — NFR-017 [Medium]: The target service availability for a production deployment is 99% or better, subject to infrastructure and third-party availability; health checks shall expose the status of critical dependencies.
- **Reliability/Consistency** — NFR-018 [High]: The system shall avoid duplicate order creation caused by payment retries or repeated client requests, and order/payment/inventory state transitions shall remain consistent across successful transactions (NFR-019 [High]).
- **Correctness** — NFR-020 [High]: Financial calculations (totals, commission, settlement, refunds) shall be deterministic and reproducible from stored order data at any later time.
- **Scalability** — NFR-021 [Medium]: The architecture shall allow growth in users, vendors, products and orders without major functional redesign; application instances should be stateless so horizontal scaling can be introduced if required (NFR-022 [Medium]).
- **Usability** — NFR-023 [Medium]: The web interface shall be responsive across common desktop, tablet and mobile viewport sizes, and navigation, validation and error messages shall be understandable to the target user classes in Section 2.3.
- **Accessibility** — NFR-024 [Medium]: Forms and interactive components should follow common accessibility practices — labels, readable contrast, keyboard-friendly interaction — aligned with WCAG 2.1 AA where feasible.
- **Maintainability** — NFR-025 [Medium]: The codebase shall use modular organization and consistent naming/coding conventions, and APIs/important modules shall be documented sufficiently for another team member to maintain them (NFR-026 [Medium]).
- **Portability** — NFR-027 [Low]: The application shall run in containerized form on any Dockercompatible host without code changes; environment-specific values shall be externalized so the same build can be promoted across environments (NFR-028 [Medium]).

- **Interoperability** — NFR-029 [Medium]: External integrations (payment, email, storage, shipping) shall be abstracted behind service interfaces so a provider can be replaced with minimal impact on business logic.
- **Testability** — NFR-030 [Medium]: Business-logic modules shall be structured to support automated unit and integration testing independent of the UI (see Section 11 of the accompanying test plan).
- **Robustness** — NFR-031 [Medium]: Failures in an external integration shall produce a controlled error state and shall not corrupt core order, payment or inventory data.

© 2026 Project Team. Prepared for academic software engineering coursework.

- **Compatibility** — NFR-032 [Medium]: The application shall support current versions of Chrome, Edge, Firefox and Safari within the supported deployment scope.

5.5 Business Rules

The following operating principles constrain system behaviour independent of any single feature and shall be enforced consistently wherever the related data or workflow is touched.

1. BR-01: Only approved vendors may publish active products.
2. BR-02: A vendor may manage only products and vendor orders assigned to that vendor.
3. BR-03: Customers cannot purchase quantities greater than available inventory at the time of order confirmation.
4. BR-04: An order shall not be treated as successfully paid until the configured payment verification process indicates success.
5. BR-05: A multi-vendor cart may result in multiple vendor-specific sub-orders under one parent customer order.
6. BR-06: Order cancellation is permitted only for eligible order states according to configured policy.
7. BR-07: Only customers with an eligible purchase should be permitted to review/rate the associated product.
8. BR-08: Coupons must satisfy configured validity, eligibility, expiry and usage rules.
9. BR-09: Platform commission shall be calculated using the configured commission policy.
10. BR-10: Vendor settlement shall represent the vendor share after applicable platform deductions.
11. BR-11: Out-of-stock products shall not be orderable.
12. BR-12: Deactivated or rejected products shall not be shown as purchasable active listings.
13. BR-13: Administrator permissions shall supersede normal customer and vendor management permissions.
14. BR-14: Important business state changes should be auditable.
15. BR-15: A refund shall never exceed the original captured payment amount for the associated order.
16. BR-16: A coupon discount shall never reduce the payable order total below zero.

6. Other Requirements

6.1 Database Requirements

The relational schema shall implement the entities and relationships defined in Appendix B.3 (EntityRelationship Diagram). Referential integrity shall be enforced through primary/foreign key constraints; order line items shall preserve the purchased price/quantity at time of sale independent of later product changes; financial records (payments, refunds, commissions, settlements) shall be immutable once finalized, with corrections modelled as new, linked records rather than edits in place; and inventory decrement/restore operations shall be wrapped in database transactions to prevent race conditions under concurrent orders.

© 2026 Project Team. Prepared for academic software engineering coursework.

6.2 Internationalization Requirements

Version 1.0 targets a single language (English) and a single currency, configured at deployment time. All user-facing text shall nonetheless be isolated from code (e.g., in resource/message files) rather than hard-coded, so that additional languages and currencies can be added in a later release (see Appendix C, Item TBD-6) without a structural rewrite. Dates and currency amounts shall be formatted consistently using a single configurable locale.

6.3 Legal, Compliance and Licensing Requirements

- The platform shall avoid storing prohibited raw payment card data and shall rely on a PCI-DSScompliant payment gateway for card handling (see NFR-015).
- Customer personal data (name, email, address) shall be collected only as needed to fulfil orders and shall be handled consistent with a documented privacy notice.
- Vendor-facing terms shall state that vendors are responsible for the accuracy and legality of the products, descriptions and prices they list.
- All third-party open-source libraries and frameworks used (React, Node.js, PostgreSQL, etc.) shall be used under their respective licences; a consolidated third-party licence list shall be maintained alongside the codebase.
- As an academic project, the system is not required to obtain formal regulatory certification, but design decisions should not knowingly contradict standard e-commerce consumer-protection principles (clear pricing, honest availability, accessible cancellation/refund process).

6.4 Reuse Objectives

Where practical, the project should reuse established, well-maintained open-source libraries (ORM, authentication, UI component libraries) rather than writing custom implementations of solved problems, to focus team effort on marketplace-specific business logic. Reusable modules developed for this project (e.g., the RBAC middleware, the payment-abstraction layer) should be structured so they could be extracted for reuse in a future related project.

Appendix A: Glossary

Term	Definition
SRS	Software Requirements Specification
RBAC	Role-Based Access Control — an authorization model in which permissions depend on assigned roles
API	Application Programming Interface
REST	Representational State Transfer, an architectural style for web APIs
SKU	Stock Keeping Unit
UAT	User Acceptance Testing

© 2026 Project Team. Prepared for academic software engineering coursework.

CI/CD	Continuous Integration / Continuous Delivery
JWT	JSON Web Token
MFA	Multi-Factor Authentication
ORM	Object-Relational Mapping
ERD	Entity-Relationship Diagram
DFD	Data Flow Diagram
KYC	Know Your Customer
Marketplace	A platform connecting multiple independent sellers with customers
Vendor	A seller approved to list and sell products on the platform
Parent Order	The customer-level order representing one checkout session that may contain items from multiple vendors
Vendor Order	The vendor-specific subset of a parent order used for seller fulfilment
Inventory	The quantity of a product currently available for sale
Settlement	Financial amount owed/processed to a vendor after commission and applicable adjustments
Commission	Platform fee retained from vendor sales
Coupon	A configured discount code applied at checkout subject to eligibility rules
Soft Delete	Marking a record inactive/deactivated rather than physically deleting it, to preserve historical data

Appendix B: Analysis Models

This appendix presents the pertinent analysis models supporting the requirements in this SRS: a system architecture (component) diagram, a use case diagram, an entity-relationship diagram, and an order status state diagram. These are conceptual models intended to guide detailed design; they are not exhaustive UML specifications.

B.1 System Architecture / Component Diagram

Figure 1 shows the layered architecture described in Section 2.1 and 3.3: client devices communicate over HTTPS with the presentation layer, which calls the API/application layer, which enforces business

© 2026 Project Team. Prepared for academic software engineering coursework.

rules before reaching the data access and data layers; external services are reached through an integration layer.

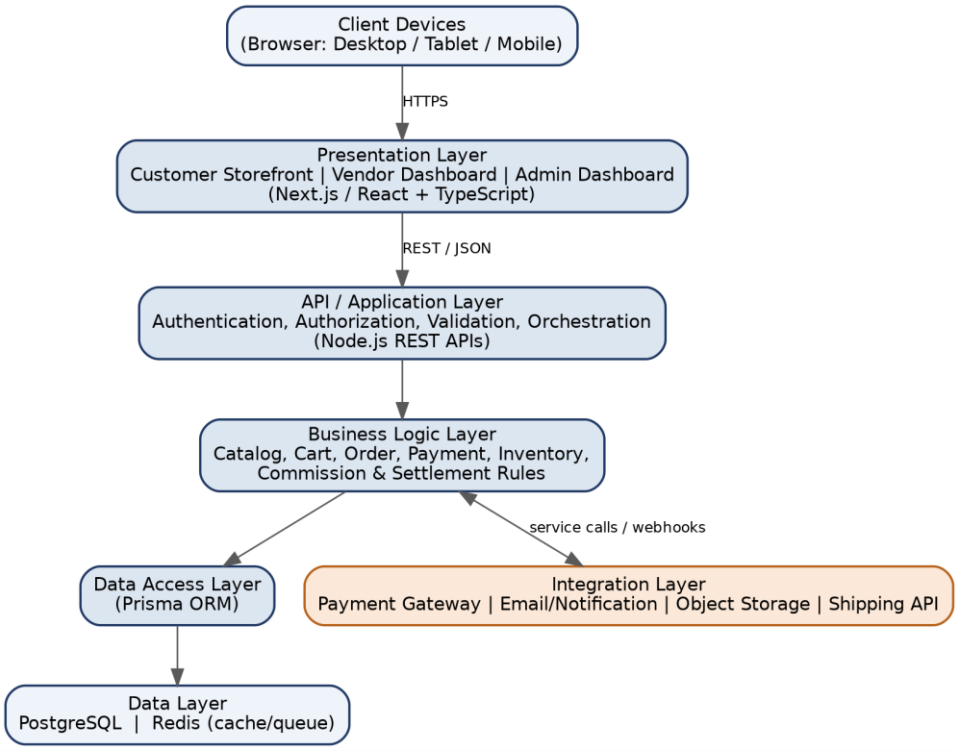


Figure 1. System architecture / component diagram.

B.2 Use Case Diagram

Figure 2 summarizes the primary actors (Customer, Vendor, Administrator) and their major use cases, corresponding to the functional requirements in Section 4.

© 2026 Project Team. Prepared for academic software engineering coursework.



Figure 2. Use case diagram for the three primary actors.

B.3 Entity-Relationship Diagram

Figure 3 shows the core entities identified in Section 6.1 (Database Requirements) and their key relationships, forming the basis for the relational schema.

© 2026 Project Team. Prepared for academic software engineering coursework.

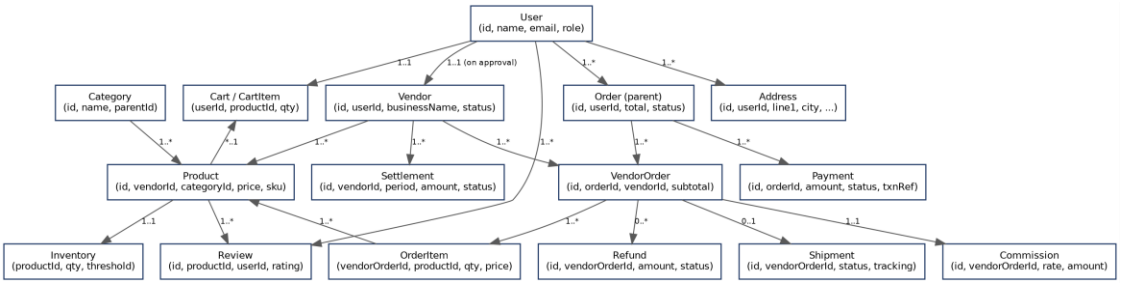


Figure 3. Entity-relationship diagram of core marketplace entities.

B.4 Order Status State Diagram

Figure 4 shows the permitted order-status transitions referenced by REQ-ORD-04, REQ-ORD-07 and BR-06.

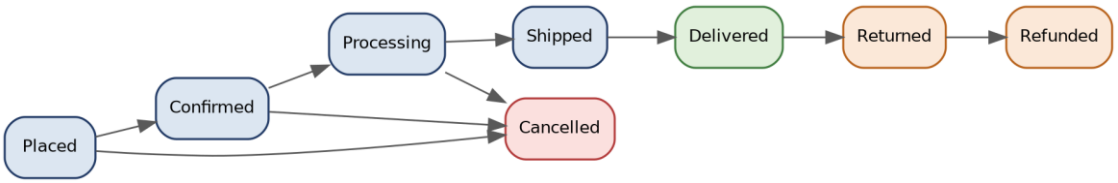


Figure 4. Order status state diagram.

Appendix C: To Be Determined List

The following items remain open at the time of this baseline and shall be tracked to closure before or during the relevant implementation phase.

#	TBD Item	Related Section(s)

© 2026 Project Team. Prepared for academic software engineering coursework.

TBD-1	Final choice of payment gateway (Razorpay vs. Stripe) and confirmation of sandbox account availability for the team.	3.3, 4.6
TBD-2	Exact commission rate(s) and whether commission varies by category or vendor tier.	4.11, 8
TBD-3	Exact eligibility window (in days) for returns/refunds and which product categories, if any, are non-returnable.	4.9, 9.6
TBD-4	Whether MFA (REQ-AUTH-09) is enforced for all vendors/administrators or offered as an opt-in control at launch.	4.1, 5.3
TBD-5	Target cloud hosting provider and environment topology (single-region vs. multi-region) for production deployment.	2.4, 12
TBD-6	Scope and timeline for multi-language/multi-currency support beyond the Version 1.0 single-locale baseline.	6.2
TBD-7	Whether a native mobile application is pursued in a future release, and if so, the camera/hardware interfaces it would require.	3.2, Future Scope
TBD-8	Final low-stock alert threshold — platform-wide default vs. fully vendorconfigurable.	4.8 (REQ-INV-06)

Future Scope (Beyond Version 1.0)

The following capabilities are recognized as valuable extensions but are explicitly excluded from Version 1.0 to keep scope realistic for the project timeline:

- AI-powered personalized recommendations and a conversational shopping assistant
- Fraud and anomaly detection
- Advanced seller advertising and sponsored listings
- Real-time courier GPS tracking
- Native mobile applications
- Multi-language and multi-currency marketplace
- Automated KYC/identity verification and automated bank settlements
- Event-driven/microservices architecture at larger scale
- Advanced analytics and demand forecasting
- Loyalty, rewards and subscription programs

© 2026 Project Team. Prepared for academic software engineering coursework.